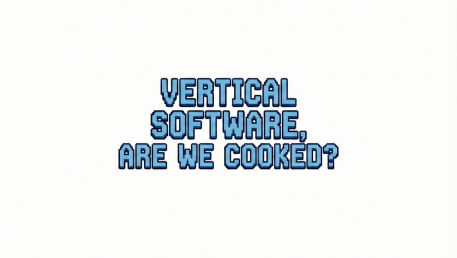
FRED TALKS

3rd May 2026

CONTENTS

10 Years Building Vertical Software: My Perspective on the Selloff

NICOLAS BUSTAMANTE · 22 FEB 2026 · SOURCE



Nicolas Bustamante

In the past few weeks, nearly $1 trillion was wiped from software and services stocks. FactSet dropped from a $20B peak to under $8B. S&P Global lost 30% in weeks. Thomson Reuters shed almost half its market cap in a year. The S&P 500 Software & Services Index, 140 companies, fell 20% year to date.

Last week, Anthropic released industry-specific plugins for Claude's Cowork which is an AI agent designed specifically for knowledge workers that can autonomously handle complex research, analysis, and document workflows.

Wall Street called it a panic. I've spent the last decade building vertical SaaS. First Doctrine, now the largest legal information platform in Europe (competing with LexisNexis, Westlaw etc) and then Fintool, an AI-powered equity research platform in the US that competes with Bloomberg, FactSet, and S&P Global today.

I built the kind of software that LLMs are now threatening. And I'm now building the kind of software that's doing the threatening. I've been on both sides of this disruption.

Here's what I see: LLMs are systematically dismantling the moats that made vertical software defensible. But not all of them. The result is a redrawing of what makes vertical software valuable and the multiple it deserves.

In this article:

- The ten moats that made vertical software defensible, and what LLMs do to each

- Why the market selloff is structurally justified but temporally exaggerated

- What the real threat actually is (it's not what you think)

- What replaces vertical software

- What comes next for the $500B+ vertical software industry

The Ten Moats of Vertical Software (and What LLMs Do to Each)

Vertical software is software built for a specific industry. Bloomberg for finance. LexisNexis for legal. Epic for healthcare. Procore for construction. Veeva for life sciences, etc.

These companies share a defining characteristic: they charge a lot and customers rarely leave. FactSet charges $15,000+ per user per year. Bloomberg Terminal costs $25,000 per seat. LexisNexis charges law firms thousands per month. And retention rates hover around 95%.

I would say that there are ten distinct moats. LLMs are attacking some of them while leaving others intact. Understanding which is which is the entire game.

THE TEN MOATS	
DESTROYED / WEAKENED	INTACT / STRONGER
1. Learned Interfaces x	6. Proprietary Data +
2. Business Logic x	7. Regulatory Lock-in +
3. Public Data Access x	8. Network Effects +
4. Talent Scarcity x	9. Transaction Embedding +
5. Bundling x	10. System of Record ~

Learned Interfaces → Destroyed

A Bloomberg Terminal user has spent years learning keyboard shortcuts, function codes, and navigation patterns. GP, FLDS, GIP, FA, BQ. These aren't intuitive. They're a language. And once you speak it fluently, switching to another platform means becoming illiterate again.

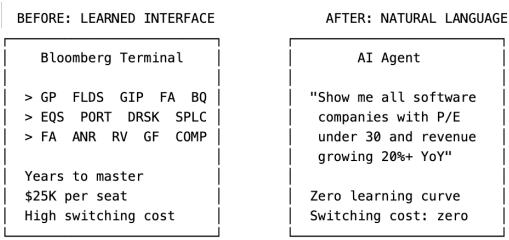
I've heard it countless times. "We're a FactSet shop." "We're a Lexis firm." "We're a Bloomberg house." These aren't statements about data quality or feature sets. They're statements about software muscle memory. People have spent a decade learning the tool. That investment isn't transferable.

This was the most underappreciated moat. Knowledge workers pay to not relearn a workflow they've spent a decade mastering. The interface IS a big part of the value prop.

I lived this at Doctrine. We had a team of designers and a small army of customer success managers whose entire job was onboarding lawyers onto our interface. Every UI change was a project: user research, design sprints, careful rollouts, handholding. We'd spend weeks on a faceted search filter redesign because lawyers had built muscle memory around the old one. The interface wasn't a feature. It was the product. And maintaining it was one of our biggest cost centers.

At Fintool, we have no onboarding. No CSMs teaching people how to navigate the product. Our users type what they want in plain English and get an answer. There is no interface to learn. That entire cost center, the designers, the CSMs, the UI change management, it just doesn't exist. The chat interface absorbed all those scaffoldings.

LLMs collapse all proprietary interfaces into one Chat.



Consider what a financial analyst does today on a Bloomberg Terminal. They navigate to the equity screening function. Set parameters using specialized syntax. Export results. Switch to the DCF model builder. Input assumptions. Run sensitivity analysis. Export to Excel. Build a presentation.

Each step requires learned interface knowledge. Each step reinforces switching costs.

Now consider what the same analyst does with an LLM agent:

"Show me all software companies with over $1B market cap, P/E under 30, and revenue growing over 20% year over year. Build a DCF model for the top 5. Run sensitivity analysis on discount rate and terminal growth."

Three sentences. No keyboard shortcuts. No function codes. No navigation. The user doesn't even know which data provider the LLM queried. They don't care.

When the interface is a natural language conversation, years of muscle memory become worthless. The switching cost that justified $25K per seat per year dissolves. For many vertical software companies, the interface was most of the value. The underlying data was licensed, public, or semi-commoditized. What justified premium pricing was the workflow built on top of that data. That's over.

2. Custom Workflows and Business Logic → Vaporized

Vertical software encodes how an industry actually works. A legal research platform doesn't just store case law. It encodes citational networks, Shepardize signals, headnote taxonomies, and the specific way a litigation associate builds a brief.

This business logic took years to build. It reflects thousands of conversations with domain experts. When I built Doctrine, the hardest part wasn't the technology. It was understanding how lawyers actually work: how they research case law, how they draft documents, how they build a litigation strategy from intake to trial. Encoding that understanding into working software was a huge part of what made vertical software valuable—and defensible.

LLMs turn all of this into a markdown file.

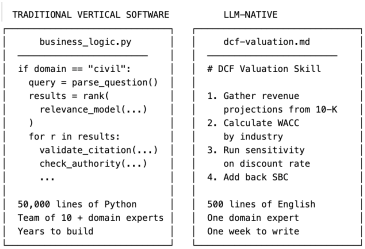
This is the most under appreciated shift and I think the most devastating long-term.

Traditional vertical software encodes business logic in code. Thousands of if/then branches, validation rules, compliance checks, approval workflows. Hardcoded by engineers over years... and not just any engineers. You need software engineers who actually understand the domain, which is rare. Finding someone who can write production code AND understands how a litigation workflow actually functions, or how a DCF model should be structured, is incredibly hard. Modifying this business logic required development cycles, QA, deployment.

Let me give you a concrete example from my own experience.

At Doctrine, we built a legal research workflow that helped lawyers find relevant case law for a given legal question. The system needed to understand the legal domain (civil vs. criminal vs. administrative), parse the question into searchable concepts, query across multiple court databases, rank results by relevance and authority, and present them with proper citational context. Building this took a team of engineers and legal experts over several years. The business logic was spread across thousands of lines of Python, custom ranking algorithms, and hand-tuned relevance models. Every modification required engineering sprints, code review, testing, and deployment.

At Fintool, we have a DCF valuation skill. It tells an LLM agent how to do a discounted cash flow analysis: which data to gather, how to calculate WACC by industry, what assumptions to validate, how to run sensitivity analysis, when to add back stock-based compensation. It's a markdown file. Writing it took a week. Updating it takes minutes. A portfolio manager who's done 500 DCF valuations can encode their entire methodology without writing a single line of code.



> Years of engineering versus one week of writing. That's the shift.

And it's not just speed. The markdown skill is better in important ways. It's readable by anyone. It's auditable. It can be customized per user (our customers write their own skills). And it gets better automatically as the underlying model improves, without us touching a line of code.

Business logic is migrating from code written by specialized engineers to markdown files that anyone with domain expertise can write. The accumulated business logic that took vertical software companies a decade to build can now be replicated in weeks. The workflow moat is eroding very fast.

3. Public Data Access → Commoditized

A massive portion of vertical software's value proposition was making hard-to-access data easy to query. FactSet makes SEC filings searchable. LexisNexis makes case law searchable. These are genuine services. SEC filings are technically public, but try reading a 200-page 10-K in raw HTML. The structure is inconsistent across companies. The accounting terminology is dense. Extracting the actual numbers you need requires parsing nested tables, following footnote references, reconciling restated figures.

Before LLMs, accessing this public data required specialized software and significant engineering scaffolding. Companies like FactSet built thousands of parsers, one for each filing type, each company's idiosyncratic formatting. Armies of engineers maintained these parsers as formats changed. The code to turn a raw SEC filing into queryable data was a genuine competitive advantage.

At Doctrine, this was also a lot of work. We built NLP pipelines for different case laws: named entity recognition to extract judges, courts, legal concepts. Dedicated ML models to classify decisions by legal domain. Custom parsers for every court, each with its own formatting quirks. We had engineers who spent years building and maintaining this scaffolding. It was genuinely impressive technology, and it was a real moat because replicating it meant years of work.

At Fintool, we built none of that. Zero NER. Zero custom parsers. Zero industry-specific classifiers. Why? Because frontier models already know how to navigate a 10-K. They know that Home Depot's ticker is HD. They understand the difference between GAAP and non-GAAP revenue. They can parse a nested table of segment disclosures without being taught the schema. The parsing infrastructure that took Doctrine years to build is now a commodity capability that comes free with the model.

LLMs make this trivial. Frontier models already know how to parse SEC filings from their training data. They understand the structure of a 10-K, where to find revenue recognition policies, how to reconcile GAAP and non-GAAP figures. You don't need to build a parser. The model IS the parser. Feed it a 10-K and it can answer any question about it. Feed it the entire corpus of federal case law and it can find relevant precedent.

The parsing, structuring, and querying that vertical software spent decades building is now a commodity capability baked into the foundation models themselves. The data isn't worthless. But the "making it searchable" layer, which is where a lot of the value and pricing power lived, is collapsing.

4. Talent Scarcity → Inverted

Building vertical software requires people who understand both the domain and the technology. Finding an engineer who can write production code AND understands how credit derivatives are structured is extremely rare. This scarcity created a natural barrier to entry that historically limited the number of serious competitors in any vertical.

LLMs flip this moat entirely.

At Doctrine, hiring was brutal. We didn't just need good engineers. We needed engineers who could understand legal reasoning: how precedent works, how jurisdictions interact, what grounds for appeal to the supreme court look like. These people barely existed. So we built our own. Every week, we held internal lectures where lawyers taught engineers how the legal system actually worked. It took months before a new engineer was productive. The talent scarcity was a genuine barrier, not just for us, but for anyone trying to compete with us.

At Fintool, we don't do any of that. Our domain experts (portfolio managers, analysts) write their methodology directly into markdown skill files. They don't need to learn Python. They don't need to understand APIs. They write in plain English what a good DCF analysis looks like, and the LLM executes it. The engineering is handled by the model. The domain expertise, which was always the abundant resource, can now become software directly without the engineering bottleneck.

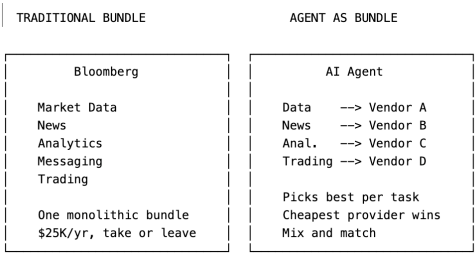
LLMs make the engineering trivially accessible, which means the scarce resource (domain expertise) is suddenly abundant in its ability to become software. This is why the barrier to entry collapses so dramatically.

5. Bundling → Weakened

Vertical software companies expand by bundling adjacent capabilities. Bloomberg started with market data, then added messaging, news, analytics, trading, and compliance. Each new module increases switching costs because customers now depend on the entire ecosystem, not just one product. S&P Global's acquisition of IHS Markit for $44B was exactly this strategy. The bundle becomes the moat.

At Doctrine, bundling was the growth strategy. We started with case law search, then added legislation, then legal news, then alerts, then document analysis. Each module had its own UI, its own onboarding, its own customer workflows. We built elaborate dashboards where lawyers could configure watchlists, set up automated alerts on specific legal topics, manage their research folders. Every feature meant more design work, more engineering, more UI surface area. The bundle kept customers locked in because they'd built their entire workflow around our ecosystem.

LLM agents break the bundling moat because the agent IS the bundle. At Fintool, alerts are a prompt. Watchlists are a prompt. Portfolio screening is a prompt. There's no separate module for each. There's no UI to maintain. A customer says "alert me when any company in my portfolio mentions tariff risk in an earnings call" and it just works. The agent orchestrates across ten different specialized tools in a single workflow. It can pull market data from one source, news from another, run analytics through a third, and compile the results. The user never knows or cares that five different services were queried.



When the integration layer moves from the software vendor to the AI agent, the incentive to buy a bundle evaporates. Why pay Bloomberg's premium for the entire suite when an agent can cherry-pick the best (or cheapest) provider for each capability?

This doesn't mean bundling is dead overnight. The operational complexity of managing ten vendor relationships versus one is real. But the directional pressure is clear: agents make unbundling viable in ways that weren't possible before.

6. Private and Proprietary Data → Stronger

Some vertical software companies own or license data that doesn't exist anywhere else. Bloomberg collects real-time pricing data from trading desks worldwide. S&P Global owns credit ratings and proprietary analytics. Dun & Bradstreet maintains business credit files on 500M+ entities. This data was collected over decades, often through exclusive relationships. You can't just scrape it. You can't recreate it.

If your data genuinely cannot be replicated, LLMs make it MORE valuable, not less.

Bloomberg's real-time pricing data from trading desks? Can't be scraped. Can't be synthesized. Can't be licensed from a third party. In an LLM world, this data becomes the scarce input that every agent needs. Bloomberg's pricing power on proprietary data may actually increase.

S&P Global's credit ratings are similar. A credit rating isn't just data. It's an opinion backed by a regulated methodology and decades of default data. An LLM can't issue a credit rating. S&P can.

The test is simple: Can this data be obtained, licensed, or synthesized by someone else? If no, the moat holds. If yes, you're in trouble.

I've seen this play out across both companies. When we started Doctrine, the core value was organizing public French case law with layers of industry-specific scaffolding: taxonomies, citational networks, relevance rankings. But the team recognized early that public data alone

wasn't enough. About five years ago, we started building an exclusive content library: proprietary legal annotations, editorial analysis, curated commentaries that don't exist anywhere else. Today, that library is genuinely hard to replicate and it's become the real moat. The companies that survive this transition are the ones that moved from "we organize public data better" to "we own data you can't get anywhere else."

Here's what's changed: that intelligence layer used to require years of engineering. Now it's a capability that comes with the model. And even the data access itself is being commoditized. MCP (Model Context Protocol) is turning every data provider into a plug-in. Dozens of companies are already offering financial data as MCP servers that any AI agent can query. When your data is available as a Claude plugin, the "making it accessible" premium disappears.

The irony is that LLMs accelerate the bifurcation. Companies with proprietary data win bigger. Companies without it lose everything.

Here's the uncomfortable implication: if your data isn't truly unique—if it can be obtained, licensed, or synthesized elsewhere you're not safe. You're at risk of commoditization. The AI agent will own the relationship with the customer. It will be the interface users interact with, the brand they trust, the product they pay for. You become a supplier to the agent, not a vendor to the customer. This is aggregation theory playing out in real-time: the aggregator (the agent) captures the user relationship and margin, while the suppliers (data vendors) compete on price to feed the platform. If Bloomberg, FactSet, and a dozen smaller providers all offer similar market data, the agent will route to whichever is cheapest. Your pricing power evaporates. Your margins compress. You become a commodity input to someone else's product.

7. Regulatory and Compliance Lock-in → Structural

In healthcare, Epic's dominance isn't just about product quality. It's about HIPAA compliance, FDA certification, and the 18-month implementation cycles that hospitals endure. Switching EHR vendors is a multi-year, multi-million dollar project that literally risks patient safety. In financial services, compliance requirements create similar lock-in. Audit trails, regulatory reporting, data retention policies. All baked into the software.

HIPAA doesn't care about LLMs. FDA certification doesn't get easier because GPT-5 exists. SOX compliance requirements don't change because Anthropic released a new plugin.

Epic's dominance in healthcare EHR is fundamentally a regulatory moat. The 18-month implementation cycles, the compliance certifications, the integration with hospital billing systems. None of this is affected by LLMs.

In fact, regulatory requirements may slow LLM adoption in exactly the verticals where compliance lock-in is strongest. A hospital can't replace Epic with an LLM agent because the LLM agent isn't HIPAA certified, doesn't have the required audit trails, and hasn't been validated by the FDA for clinical decision support.

8. Network Effects → Sticky

Some vertical software becomes more valuable as more industry participants use it. Bloomberg's messaging function (IB chat) is the de facto communication layer for Wall Street. If every counterparty uses Bloomberg, you have to use Bloomberg. Not because of the data. Because of the network.

LLMs don't break network effects. If anything, they might make communication networks more valuable. The information flowing through these networks becomes training data, context, signal.

The same applies to any vertical software that functions as a communication layer within an industry. Veeva's network effects across pharma companies. Procore's network effects across construction stakeholders. These are sticky because the value comes from who else is on the platform, not from the interface.

9. Transaction Embedding → Durable

Some vertical software sits directly in the money flow. Payment processing for restaurants. Loan origination for banks. Claims processing for insurance companies. When you're embedded in the transaction, switching means interrupting revenue. Nobody does that voluntarily.

If your software processes payments, originates loans, or settles trades, an LLM doesn't disintermediate you. It might sit on top of you as a better interface, but the rails themselves remain essential.

Stripe isn't threatened by LLMs. Neither is FIS or Fiserv. The transaction processing layer is infrastructure, not interface.

10. System of Record Status → Threatened Long-Term

When your software is the canonical source of truth for critical business data, switching isn't just inconvenient. It's existentially risky. What if data gets corrupted during migration? What if historical records are lost? What if audit trails break?

Epic is the system of record for patient data. Salesforce is the system of record for customer relationships. These companies benefit from the asymmetry between the cost of staying (high fees) and the cost of leaving (potential data loss, operational disruption).

> LLMs don't directly threaten system of record status today. But agents are quietly building their own.

Here's what's happening: AI agents don't just query existing systems. They read your SharePoint, your Outlook, your Slack. They collect data on the user. They write detailed memory files that persist across sessions. And when they perform key actions, they store that context. Over time, the agent accumulates a richer, more complete picture of a user's work than any single system of record.

The agent's memory becomes the new source of truth. Not because anyone planned it, but because the agent is the one layer that sees everything. Salesforce sees your CRM data. Outlook sees your emails. SharePoint sees your documents. The agent sees all three, and remembers.

This doesn't happen overnight. But directionally, agents are building their own system of record from the ground up. The traditional system of record's moat weakens as the agent's contextual memory grows.

The Net Effect: Barrier to Entry Collapses

Add it all up. Five moats destroyed or weakened. Five that hold. But the five that break are the ones that kept competitors out. The ones that hold are the ones that only some incumbents have.

Before LLMs, building a credible competitor to Bloomberg or LexisNexis required hundreds of engineers who understand the domain, years of development time, massive data licensing deals, sales teams that can sell to conservative enterprises, and regulatory certifications. The result: most verticals had 2-3 serious competitors.

After LLMs, a small team with frontier model APIs, domain expertise, and good data pipelines can build a product that handles 80% of what a vertical software does within months. I know this because I've done it. Fintool was built by a team of six. We serve hedge funds that previously relied exclusively on Bloomberg and FactSet. Not because we have better data. Because our AI agent delivers answers faster and more intuitively than a terminal/workstation that requires years of training to master.

The critical insight is that competition doesn't increase linearly—it explodes combinatorially. You don't go from 3 incumbents to 4. You go from 3 to 300. And that's what craters pricing power. Before LLMs, each vertical had 2-3 dominant players commanding premium prices because the barriers to entry were insurmountable. That math changes completely when 50 AI-native startups can offer 80% of the capability at 20% of the price.

The Nuance: This is a Multi-Year Transition, Not an Overnight Collapse

Here's where I think the market is getting the timing wrong, even if the direction is right.

Enterprise Revenue Doesn't Disappear Overnight

FactSet's clients are on multi-year contracts. Bloomberg Terminal contracts are typically 2-year minimums. These contracts don't evaporate because Anthropic released a plugin.

Enterprise procurement cycles are measured in quarters and years, not days. A $50B hedge fund isn't going to rip out S&P Global CapIQ tomorrow because Claude can query SEC filings. They'll evaluate alternatives over 12-18 months. They'll run pilot programs. They'll negotiate contract terms. They'll wait for their existing contracts to expire.

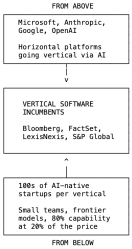
The revenue cliff is real but it's a slope, not a cliff. Current revenue is largely locked in for the next 12-24 months.

But here's the thing the market already understands: you don't need revenue to decline for the stock to crash. You need the multiple to compress. A financial data company that traded at 15x revenue when it had pricing power and 95% retention might trade at 6x revenue when the market believes both are eroding. Revenue stays flat. The stock drops 60%. That's exactly what's happening to some companies right now.

The market isn't pricing in a revenue collapse. It's pricing in the end of the premium multiple, because the moats that justified that multiple are dissolving.

The Real Threat

The real threat isn't the LLM itself. It's a pincer movement that vertical software incumbents didn't see coming.



From below, hundreds of AI-native startups are entering every vertical. When building a credible financial data product required 200 engineers and $50M in data licensing, markets naturally consolidated to 3-4 players. When it requires 10 engineers and frontier model APIs, the market fragments violently. Competition goes from 3 to 300.

From above, horizontal platforms are going deep into vertical territory for the first time. Microsoft Copilot inside Excel now does AI-powered DCF modeling and financial statement parsing. Copilot inside Word does contract review and case law research. The horizontal tool becomes vertical through AI, not through engineering.

Anthropic is doing the same thing from the other direction. And I'm watching it up close because Fintool is an Anthropic-backed company. Claude is going all-in on vertical. And the playbook is terrifyingly simple: a general-purpose agent harness (the SDK), pluggable data access (MCP), and domain-specific skills (markdown files). That's it. That's the entire stack you need to go from horizontal to vertical. No domain engineers. No years of development.

Software is becoming headless. The interface disappears. Everything flows through the agent. What matters isn't the software anymore. It's owning the customer relationship and use cases, which means owning the agent itself.

The technology that enables vertical depth (LLMs + skills + MCP) is the same technology that lets horizontal platforms finally compete in territory they could never reach before. And this is perhaps the most existential threat for vertical software: horizontal B2B players like Microsoft aren't just dabbling in vertical anymore they're extending aggressively into it because it's now easier than ever, and because they need to own the use cases and workflows to stay relevant in an AI-first world.

A Framework for Assessing Risk

Not all vertical software is equally exposed. Here's how I think about which categories survive and which don't.

High Risk: The Search Layer

If your primary value is making data searchable and accessible through a specialized interface, and the underlying data is public or licensable, you are in serious trouble. This includes financial data terminals built on licensed exchange data, legal research platforms built on public case law, patent search tools, and any vertical where the product is essentially "we built a better search engine for your industry's data."

These companies traded at 15-20x revenue because of interface lock-in and limited competition. Both are evaporating. Think of the financial data providers that have lost 40-60% of their market cap in the last year. The market is right to reprice them.

Medium Risk: The Mixed Portfolio

Many vertical software companies have a mix of defensible and exposed business lines. A company might have a genuinely proprietary ratings business alongside a data analytics segment that's mostly repackaged public information. Or an index licensing business (embedded in the transaction, very defensible) alongside a research platform (pure search layer, very exposed).

The stock declines in this category (20-30%) reflect the market's uncertainty about which segments dominate the valuation. The key question: what percentage of revenue comes from moats that LLMs can't touch?

Lower Risk: Regulatory Fortresses

If your moat is regulatory certification, compliance infrastructure, and deep integration with mission-critical workflows, LLMs are barely relevant to your competitive position in the medium term. Healthcare EHR systems with HIPAA compliance and FDA validation. Life sciences platforms with regulatory lock-in. Financial compliance and reporting infrastructure.

These companies may even benefit from AI disruption elsewhere, as customers consolidate around the vendors they trust for regulated workflows while switching away from the vendors they used for information retrieval.

The Test

For any vertical software company, ask three questions:

1. Is the data proprietary? If yes, the moat holds. If no, the accessibility layer is collapsing.

2. Is there regulatory lock-in? If yes, LLMs don't change the switching cost equation. If no, switching costs are primarily interface-driven and dissolving.

3. Is the software embedded in the transaction? If yes, LLMs sit on top of you, not instead of you. If no, you're replaceable.

Zero "yes" answers: high risk. One: medium risk. Two or three: you're probably fine.

Company	Propriet. Data?	Regulatory Lock-in?	Transaction Embedded?	Risk Level
Bloomberg	Yes	No	Partial	Medium
S&P Global	Yes	Partial	Yes	Low
Epic (Healthcare)	Yes	Yes	Yes	Low
FactSet	No	No	No	HIGH
LexisNexis (search)	Partial	No	No	HIGH
Veeva	Yes	Yes	Yes	Low
Stripe / FIS	No	Partial	Yes	Low

What I've Learned Building on Both Sides

When I built Doctrine starting in 2016, one of the moat was the interface. We built beautiful search experiences over case law. Lawyers loved it because it was faster and more intuitive than anything else on the market. Most of the data was public but our interface and search made it accessible. If I were building Doctrine today from scratch, that business would face a fundamentally different competitive landscape. An LLM agent can query case law as effectively as our interface could.

That realization is exactly why I built Fintool differently. We don't compete on interface. We compete on the quality of our AI agent's output, on the proprietary skills and data pipelines we've built, and on the trust we've earned with professional investors who rely on our accuracy for million dollar decisions. When the interface is a conversation, the product is the intelligence behind it.

The vertical SaaS reckoning isn't about all vertical software dying. It's about the market finally distinguishing between companies that own something genuinely scarce and companies that built a pretty interface over someone else's data.

A non-anthropomorphized view of LLMs

HALVAR.FLAKE · 14 SEP 2025 · SOURCE

In many discussions where questions of "alignment" or "AI safety" crop up, I am baffled by seriously intelligent people imbuing almost magical human-like powers to something that - in my mind - is just MatMul with interspersed nonlinearities.

In one of these discussions, somebody correctly called me out on the simplistic nature of this argument - "a brain is just some proteins and currents". I felt like I should explain my argument a bit more, because it feels less simplistic to me:

The space of words

The tokenization and embedding step maps individual words (or tokens) to some vectors. So let us imagine for a second that we have in front of us. A piece of text is then a path through this space - going from word to word to word, tracing a (possibly convoluted) line.

Imagine now that you label each of the "words" that form the path with a number: The last word with 1, counting forward until you hit the first word or the maximum context length . If you've ever played the game "Snake", picture something similar, but played in very high-dimensional space - you're moving forward through space with the tail getting truncated off.

The LLM takes your previous path into account, calculates probabilities for the next point to go to, and then makes a random pick into the next point according to these probabilities. An LLM instantiated with a fixed random seed is a mapping of the form .

In my mind, the paths generated by these mappings look a lot like strange attractors in dynamical systems - complicated, convoluted paths that are structured-ish.

Learning the mapping

We obtain this mapping by training it to mimic human text. For this, we use approximately all human writing we can obtain, plus corpora written by human experts on a particular topic, plus some automatically generated pieces of text in domains where we can automatically generate and validate them.

Paths to avoid

There are certain language sequences we wish to avoid - because the sequences these models generate try to mimic human speech in all it's empirical structure, but we feel that some of the things that humans have empirically written are very undesirable to be generated. We also feel that a variety of other paths should ideally not be generated, if - when interpreted by either humans or other computer systems - undesirable results arise.

We can't specify strictly in a mathematical sense which paths we would prefer not to generate, but we can provide examples and counterexamples, and we try to hence nudge the complicated learnt distribution away from them.

"Alignment" for LLMs

Alignment and safety for LLMs mean that **we should be able to quantify and bound the probability with which certain undesirable sequences are generated.** The trouble is that we largely fail at describing "undesirable" except by example, which makes calculating bounds difficult.

For a given LLM (without random seed) and sequence, it is trivial to calculate the probability of the sequence to be generated. So if we had a way of somehow summing or integrating over these probabilities, we could say with certainty "this model will generate an undesirable sequence once every N model evaluations". We can't, currently, and that sucks, but at the heart, this is the mathematical and computational problem we'd need to solve.

The surprising utility of LLMs

LLMs solve a large number of problems that could previously not be solved algorithmically. NLP (as the field was a few years ago) has largely been solved.

I can write a request in plain English to summarize a document for me and put some key datapoints from the document in a structured JSON format, and modern models will just do that. I can ask a model to generate a children's book story involving raceboats and generate illustrations, and the model will generate something that is passable. And much more, all of which would have seemed like absolute science fiction 5-6 years ago.

We're on a pretty steep improvement curve, so I expect the number of currently-intractable problems that these models can solve to keep increasing for a while.

Where anthropomorphization loses me

The moment that people ascribe properties such as "consciousness" or "ethics" or "values" or "morals" to these learnt mappings is where I tend to get lost. We are speaking about a big recurrence equation that produces a new word, and that stops producing words if we don't crank the shaft.

To me, wondering if this contraption will "wake up" is similarly bewildering as if I was to ask a computational meteorologist if he isn't afraid of his meteorological numerical calculation will "wake up".

I am baffled that the AI discussions seem to never move away from treating a function to generate sequences of words as something that resembles a human. Statements such as "an AI agent could become an insider threat so it needs monitoring" are simultaneously unsurprising (you have a randomized sequence generator fed into your shell, literally **anything** can happen!) and baffling (you talk as if you believe the dice you play with had a mind of their own and could decide to conspire against you).

Instead of saying "we cannot ensure that no harmful sequences will be generated by our function, partially because we don't know how to specify and enumerate harmful sequences", we talk about "behaviors", "ethical constraints", and "harmful actions in pursuit of their goals". All of these are anthropocentric concepts that - in my mind - do not apply to functions or other mathematical objects. And using them muddles the discussion, and our thinking about what we're doing when we create, analyze, deploy and monitor LLMs.

This muddles the public discussion. We have many historical examples of humanity ascribing bad random events to "the wrath of god(s)" (earthquakes, famines, etc.), "evil spirits" and so forth. The fact that intelligent highly educated researchers talk about these mathematical objects in anthropomorphic terms makes the technology seem mysterious, scary, and magical.

We should think in terms of "this is a function to generate sequences" and "by providing prefixes we can steer the sequence generation around in the space of words and change the probabilities for output sequences". And for every possible undesirable output sequence of a length smaller than , we can pick a context that maximizes the probability of this undesirable output sequence.

A much clearer formulation, which helps more clearly articulate the problems to solve.

Why many AI luminaries tend to anthropomorphize

Perhaps I am fighting windmills, or rather a self-selection bias: A fair number of current AI luminaries have self-selected by their belief that they might be the ones getting to AGI - "creating a god" so to speak, the creation of something like life, as good as or better than humans. You are more likely to choose this career path if you believe that it is feasible, and that current approaches might get you there. Possibly I am asking people to "please let go of the belief that you based your life around" when I am asking for an end to anthropomorphization of LLMs, which won't fly.

Why I think human consciousness isn't comparable to an LLM

The following is uncomfortably philosophical, but: In my worldview, humans are dramatically different things than a function . For hundreds of millions of years, nature generated new versions, and only a small number of these versions survived. Human thought is a poorly-understood process, involving enormously many neurons, extremely high-bandwidth input, an extremely complicated cocktail of hormones, constant monitoring of energy levels, and millions of years of harsh selection pressure.

We understand essentially nothing about it. In contrast to an LLM, given a human and a sequence of words, I cannot begin putting a probability on "will this human generate this sequence".

To repeat myself: To me, considering that any human concept such as ethics, will to survive, or fear, apply to an LLM appears similarly strange as if we were discussing the feelings of a numerical meteorology simulation.

The real issues

The function class represented by modern LLMs are very useful. Even if we never get anywhere close to AGI and just deploy the current state of technology everywhere where it might be useful, we will get a dramatically different world. LLMs might end up being similarly impactful as electrification.

My grandfather lived from 1904 to 1981, a period which encompassed moving from gas lamps to electric, the replacement of horse carriages by cars, nuclear power, transistors, all the way to computers. It also spanned two world wars, the rise of Communism and Stalinism, almost the entire lifetime of the USSR and GDR etc. The world on his birth looked nothing like the world when he died.

Navigating the dramatic changes of the next few decades while trying to avoid world wars and murderous ideologies is difficult enough without muddying our thinking.